

Laboratoire 2 : système de gestion de tickets

6GEI311 Architecture des logiciels · UQAC · Automne 2026

Équipe : Zongo T Julien (ZONT16020300) et Zongo A Razack (ZONA31010300)

I. Critique du diagramme de classes initial

Nous avons implémenté le diagramme hérité tel quel en Partie 1, défauts compris, pour garder un point de comparaison. Ses faiblesses se rangent en six grands points.

1.Écarts avec l'énoncé : Le texte confie l'assignation et la fermeture aux développeurs ; le diagramme les réserve à Admin et ne dit pas qui valide un ticket. Le cycle de vie n'apparaît nulle part. description: String ne peut contenir ni capture d'écran ni vidéo(évolution).

2.Typeage faible. status et priority sont des chaînes libres. Rien n'empêche de passer d'OUVERT à TERMINÉ, ou d'écrire « termine ». Les règles de transition finiraient toutes dans updateStatus() sous forme de if/elif : chaque nouveau statut modifierait cette méthode (Open/Closed Principle :OCP), et Ticket mêlerait données et règles (Single Responsibility Principle :SRP).

3.Données manquantes. assignTo(user) reçoit un utilisateur et ne le garde pas. Le créateur n'est pas stocké malgré l'association « creates ». addComment(comment) ne mémorise rien, et un commentaire n'a ni auteur ni date. viewAllTickets() renvoie une liste que personne ne détient.

4.Responsabilités floues. Admin.assignTicket recouvre Ticket.assignTo, et User.updateTicket recouvre Ticket.updateStatus : on ne sait pas qui applique les règles.

5.Rôles modélisés deux fois. User et Admin dupliquent name et email, et User porte en plus un attribut role, alors que les deux classes expriment déjà les rôles.

6.Notation UML. Le losange plein sur « closes » voudrait dire qu'un administrateur possède ses tickets et décide de leur durée de vie. « views » et « closes » sont des actions ponctuelles, donc des dépendances. viewTicket() retourne void : on ignore ce qu'elle produit.

II. Améliorations du diagramme et justifications

Le diagramme révisé (fichier joint) s'appuie sur quatre patrons : State pour le cycle de vie, Template Method et Factory Method pour les artefacts, Command pour l'édition et les lots(agrégation). La mention « Défaut » renvoie aux 6 grands points de la Section I.

Tableau 1 : Transitions et acteurs

OUVERT	ASSIGNÉ (Admin), FERMÉ (Admin)
ASSIGNÉ	ASSIGNÉ par réassignation (Admin), OUVERT par désassignation (Admin), VALIDATION (développeur assigné), FERMÉ (Admin)
VALIDATION	TERMINÉ (Admin), ASSIGNÉ en cas de rejet (Admin)
TERMINÉ, FERMÉ	États terminaux, aucune sortie

Toute autre demande lève `TransitionInvalideException`, quel que soit l'appelant. `ASSIGNÉ` a quatre sorties et forme une boucle avec `VALIDATION` : c'est là qu'une suite de conditions serait devenue illisible.

Tableau 2 : Changements majeurs et justifications

#Changement apporté	Justification (principe, tactique de modifiabilité)
Cycle de vie du ticket (State)	
1 <code>status</code> : String devient un état courant (composition Ticket → <code>EtatTicket</code>) et l'énumération <code>Statut</code> . <code>status</code> reste lisible comme attribut dérivé, sans setter. Défaut (1), (2)	Le statut est désormais un comportement. <code>ticket.status = ...</code> lève une erreur : on ne peut plus sauter une étape. Tactique : <i>encapsulation</i> .
2 Classe abstraite <code>EtatTicket</code> dont <code>gererTransition()</code> refuse par défaut, et cinq états concrets. Défaut (2)	Chaque état ne connaît que ses propres sorties. Un nouvel état est une nouvelle classe. Tactique : <i>augmentation de la cohésion</i> .
3 <code>updateStatus()</code> délègue à l'état courant, qui crée l'état suivant et appelle <code>setEtat()</code> . Défaut (4)	Ticket ne contient aucun test sur le statut, et un refus intervient avant toute modification.
4 <code>priority</code> : String devient l'énumération <code>Priorite</code> . Défaut (2)	Les valeurs sont fermées ; une faute de frappe est signalée par l'analyse statique.
Acteurs, rôles et données	
5 Classe abstraite <code>UtilisateurSysteme</code> ← <code>User</code> , <code>Admin</code> ; <code>role</code> disparaît ; le dépôt est injecté au constructeur. Défaut (5)	L'identité est écrite une fois (DRY : Don't Repeat Yourself) et le rôle découle de la classe. Tactique : <i>réduction du couplage</i> .
6 L'Admin pilote le cycle de vie (assigner, réassigner, désassigner, valider, fermer). Le User gère le contenu de ses tickets et soumet son travail en validation. Défaut (1), (4)	Un acteur, une raison d'agir. Un User n'a pas les méthodes de l'Admin : aucun contrôle à l'exécution, aucune méthode héritée neutralisée (Liskov).
7 <code>_assignedUser</code> et mémoire d'assignation <code>aDejaEteAssigne</code> ; l'Admin demande la transition, puis modifie la donnée. Défaut (3)	On sait qui traite le ticket. Une transition refusée ne laisse jamais un assigné incohérent.
8 <code>creator</code> est renseigné par <code>User.createTicket()</code> . Le créateur peut supprimer son ticket tant qu'il n'a jamais été assigné(choix). Défaut (3), (4)	<code>createTicket()</code> a enfin un rôle. Un ticket entré dans le circuit de traitement ne peut plus disparaître, il est conservé jusqu'à son état final.
9 Classe <code>Commentaire</code> (texte, auteur typé <code>UtilisateurSysteme</code> , date). Défaut (3)	Les échanges sont traçables, que l'auteur soit un User ou l'Admin.
10 Dépôt <code>SystemeTickets</code> , qui agrège les tickets. Défaut (3)	<code>viewAllTickets()</code> et <code>mesTickets()</code> ont une source. Tactique : <i>utilisation d'un intermédiaire</i> .

11	Attributs privés, lus par des propriétés sans setter ; pas de getX(). Défaut (4)	L'encapsulation cesse d'être une convention que le code client peut ignorer.
Description des tickets (Template Method, Factory Method)		
12	Description contient 0..* ElementDescription. traiter() enchaîne validation, chargement et préparation ; seul charger() varie. Défaut (1)	Un ticket accepte plusieurs artefacts de types différents. Le squelette commun est écrit une fois.
13	FabriqueDescription.creer(type, chemin), seul endroit qui connaît les sous-classes. Défaut (1)	Le client ne dépend que de l'abstraction (DIP). Tactique : <i>Utilisation d'un intermédiaire.</i>
Édition et lots (Command)		
14	Classe abstraite Commande et six commandes ; celles de l'Admin appellent ses méthodes. Défaut (4)	Chaque règle métier reste écrite à un seul endroit ; une commande n'est qu'un appel différé.
15	SessionOperations ← SessionEdition (un ticket, annulation, pause (durant la création), reprise((durant l'édition)) et LotAdmin (plusieurs tickets, application au mieux). Défaut (4)	Rien ne touche le ticket avant appliquer(), ce n'est qu'après ça qu'on applique les changements. Tactique : <i>différer la liaison.</i>
Notation UML		
16	Compositions (état, description, commentaires, pile de commandes), agrégation (dépôt), associations (créateur, assigné, auteur), dépendances (crée, lève, délègue), attribut dérivé /status. Défaut (6)	La nature des liens et les durées de vie sont exactes et en fonction de nos choix.

Description des tickets : chargement, bonnes pratiques et instanciation

Chaque artefact est désigné par son chemin, validé une seule fois dans la classe mère. Ce qui change d'un type à l'autre, c'est la façon de lire le fichier et le moment où on le lit :

Tableau 3: Traitement des artefacts

Type	Lecture	Moment	Ce qui est gardé en mémoire
Texte	mode texte, UTF-8	à l'ajout	tout le contenu (434 caractères dans la démo)
Image	mode binaire	à l'ajout	tous les octets (160 octets)
Vidéo	métadonnées, puis binaire	différé	la taille seule ; le contenu attend chargerContenuReel()

Bonnes pratiques : Template Method fixe l'ordre du traitement dans traiter(), marquée @final. Factory Method isole la construction. Le polymorphisme évite tout isinstance dans Description.afficher(). Côté principes : OCP (un type audio demanderait une sous-classe et une ligne dans la fabrique), DIP (le client manipule ElementDescription), ISP (chargerContenuReel() n'existe que sur la vidéo). Le chargement différé de la vidéo applique la tactique « différer la liaison ».

Instanciation : Le code client n'écrit jamais DescriptionVideo(...) : il appelle

FabriqueDescription.creer("video", chemin) et reçoit un ElementDescription. La fabrique construit sans charger. Dans une session d'édition, l'élément est traité dès son ajout au brouillon : un chemin invalide est refusé tout de suite, si bien que l'application de la session ne peut plus échouer pour cette raison. Un type inconnu lève ArtefactInvalideException. Les états, eux, sont instanciés par l'état précédent au moment de la transition, un objet par transition.

III. Autres améliorations du code et de la conception

Contrairement au texte, qui confie l'assignation aux développeurs, nous avons choisi avec le client une gouvernance centralisée : l'Admin affecte les tickets et prononce leur clôture, et le développeur assigné garde la main sur ce que le texte lui attribue explicitement, soumettre son travail en validation. Chaque changement d'état a ainsi un responsable identifiable.

Autres interprétations : L'assignation étant permise « à tout moment », nous acceptons la réassignation directe, la désassignation et le rejet d'une validation. Réassigner ne passe pas par OUVERT, sinon le ticket garderait la trace d'une étape que personne n'a demandée. La fermeture n'est permise que depuis OUVERT ou ASSIGNÉ : en VALIDATION, le travail est fait, et il reste à le valider ou à le rejeter. FERMÉ et TERMINÉ restent distincts, parce qu'un ticket écarté et un ticket résolu ne doivent pas se confondre dans viewAllTickets(). Enfin, la mise en contexte évoque un export PDF que la Partie 2 ne redemande pas ; nous ne l'avons pas implémenté.

Intégrité : Une transition refusée lève l'exception avant toute modification. Le statut ne suffit pas à savoir si un ticket a déjà été assigné : un ticket désassigné redevient OUVERT, comme un ticket neuf. D'où la mémoire aDejaEteAssigne, jamais remise à faux, qui interdit sa suppression.

Édition et lots : Les modifications d'une session restent en attente jusqu'à appliquer() qui est une validation finale. Annuler revient donc à retirer une commande de la pile, et aucune commande n'a besoin d'une méthode annuler(). Un lot Admin s'applique au mieux : ses opérations visent des tickets indépendants, un refus ne bloque pas les suivantes, et chaque refus est consigné avec son motif. Dans la démonstration, le lot du scénario P produit 1 succès et 3 refus, dont une assignation visant un ticket supprimé entre la préparation et l'application.

Code et démonstration : Aucune dépendance externe, un fichier par classe, six paquets. Les états se référencent mutuellement : chacun importe ses successeurs à l'intérieur de gererTransition(), et les imports utiles aux seules annotations passent sous TYPE_CHECKING. main.py déroule 17 scénarios (A à Q) ; chaque refus attendu est capturé, et le programme vérifie que le ticket est resté intact. La Partie 1, fidèle au diagramme initial, est conservée comme référence.

IV. Leçons apprises

Voir le défaut avant de le corriger : En Partie 1, rien n'empêche de passer d'OUVERT à TERMINÉ. En Partie 2, la même demande lève une exception, et ticket.status = ... aussi. Avoir écrit la mauvaise version d'abord a rendu chaque critique de la Section I facile et vérifiable.

Le principe ouvert/fermé a ses limites : Ajouter un état oblige à modifier l'état qui doit y mener, et un nouveau type d'artefact coûte une ligne dans la fabrique. Les patrons ne suppriment pas la modification, ils la ramènent à un endroit qu'on sait retrouver.

Relire l'énoncé contre le modèle : Notre première version interdisait la réassignation, le client a voulu qu'on le fasse maintenant. La correction n'a touché que deux classes d'état. De même, Pylance a signalé qu'un Admin ne pouvait pas être l'auteur d'un commentaire typé User : l'erreur était dans le modèle, pas dans le code.

L'essentiel : Toujours revenir vers le client pour savoir ses besoins profonds qu'il aura mal ou pas assez explicité dans un cahier de charges par exemple.